

Course 5: Hardware Integration

USP Training Series

USP Development Team

Universal Software Platform

November 21, 2025

Course Overview

- 1 Introduction
- 2 Device Tree Configuration
- 3 TX Power Calibration
- 4 HAL Porting
- 5 Custom Board Integration
- 6 Hands-On Labs
- 7 Summary

Course 5: Hardware Integration

Learning Objectives

- Configure device tree for custom hardware
- Calibrate TX power for regulatory compliance
- Port HAL to new MCU platforms
- Adapt USP to custom board designs
- Integrate custom sensors and peripherals

Hands-On Labs

- Lab 5.1: Device Tree Configuration
- Lab 5.2: TX Power Calibration
- Lab 5.3: Custom Board HAL Port

Prerequisites

- Completion of Courses 1-4
- Understanding of Zephyr RTOS device tree
- Familiarity with GPIO, SPI, I2C protocols
- Knowledge of LR1120/LR1121 radio configuration
- Access to test equipment (spectrum analyzer, power meter)

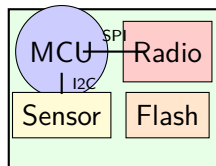
Hardware Required

Custom board or development kit different from Xiao nRF54L15

Why Hardware Integration Matters

Production Deployment:

- Custom PCB designs
- Different MCU/radio combinations
- Cost optimization
- Form factor requirements
- Specific peripheral needs



Custom Board

Regulatory Compliance:

- TX power accuracy
- Frequency accuracy
- Spurious emissions
- Certification testing

Zephyr Device Tree Overview

What is Device Tree?

Device Tree (DT) is a hardware description format used by Zephyr to configure:

- GPIO pin assignments
- SPI/I2C bus configuration
- Peripheral properties
- Board-specific settings

Key Files:

- `boards/arm/myboard/myboard.dts` – Main board definition
- `boards/arm/myboard/myboard.overlay` – Application overlay
- `dts/bindings/` – Device bindings (schema)

Syntax:

- Hierarchical node structure
- Properties: `property = <value>.`

Device Tree Structure

```
/ {  
    model = "My Custom Board";  
    compatible = "vendor,myboard";  
  
    chosen {  
        zephyr,console = &uart0;  
        zephyr,shell-uart = &uart0;  
    };  
  
    leds {  
        compatible = "gpio-leds";  
        led0: led_0 {  
            gpios = <&gpio0 13 GPIO_ACTIVE_LOW>;  
            label = "Status LED";  
        };  
    };  
  
    buttons {  
        compatible = "gpio-keys";
```

Configure SPI Bus for LR1120

```
&spi2 {
    compatible = "nordic,nrf-spim";
    status = "okay";
    cs-gpios = <&gpio0 8 GPIO_ACTIVE_LOW>;

    lr1120: lr1120@0 {
        compatible = "semtech,lr1120";
        reg = <0>;
        spi-max-frequency = <16000000>;

        reset-gpios = <&gpio0 14 GPIO_ACTIVE_LOW>;
        busy-gpios = <&gpio0 15 GPIO_ACTIVE_HIGH>;
        dio1-gpios = <&gpio0 28 GPIO_ACTIVE_HIGH>;

        // TX power configuration
        tx-power-offset = <2>; // dB
        antenna-gain = <0>; // dBi

        // Frequency calibration
```


Device Tree Overlays

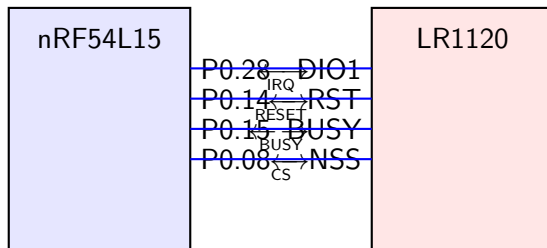
What are Overlays?

Overlays allow application-specific modifications without changing base board definition.

```
// Add sensor on I2C bus
&i2c0 {
    bme680: bme680@76 {
        compatible = "bosch,bme680";
        reg = <0x76>;
        label = "BME680";
    };
};

// Disable unused peripherals to save power
&uart1 {
    status = "disabled";
};
```

Device Tree Pin Assignment



Pin Assignment Considerations:

- Use GPIO pins that support required functions
- Avoid conflicts with other peripherals
- Consider power domain and voltage levels
- Check MCU datasheet for restrictions

Why TX Power Calibration?

Regulatory Requirements

- EU868: Maximum +14 dBm EIRP (typical)
- US915: Maximum +30 dBm EIRP
- AS923: Maximum +16 dBm EIRP
- Varies by frequency sub-band

System Factors Affecting TX Power:

- 1 Radio output power (LR1120: -17 to +22 dBm)
- 2 PCB trace losses (typically 0.5-1 dB)
- 3 Antenna matching network losses (0.5-2 dB)
- 4 Antenna gain (0 to +3 dBi typical)

$$\text{EIRP} = P_{\text{radio}} - L_{\text{traces}} - L_{\text{matching}} + G_{\text{antenna}}$$

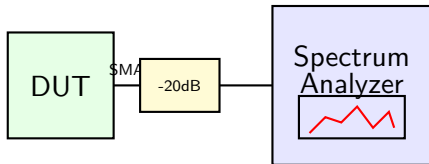
TX Power Calibration Setup

Required Equipment:

- Spectrum analyzer
- RF power meter
- 50Ω dummy load
- SMA cables
- RF attenuators (20-30 dB)

Procedure:

- 1 Set radio to CW mode
- 2 Measure output power
- 3 Calculate offset
- 4 Update device tree
- 5 Verify compliance



Measure TX Power

```
#include <lr11xx_radio.h>
#include <lr11xx_system.h>

void tx_power_calibration_mode(void) {
    // Set to CW (Continuous Wave) mode
    lr11xx_radio_set_rf_freq(868100000);    // 868.1 MHz

    // Set PA config
    lr11xx_radio_pa_cfg_t pa_cfg = {
        .pa_sel = LR11XX_RADIO_PA_SEL_HP,
        .pa_reg_supply =
            LR11XX_RADIO_PA_REG_SUPPLY_VBAT,
        .pa_duty_cycle = 0x04,
        .pa_hp_sel = 0x07,
    };
    lr11xx_radio_set_pa_cfg(&pa_cfg);

    // Set output power (register value)
    lr11xx_radio_set_tx_params(14,
```

TX Power Offset Calculation

Example Measurement

- Radio configured for: +14 dBm
- Measured output (with 20 dB attenuator): -7.5 dBm
- Actual output: $-7.5 + 20 = +12.5$ dBm
- Offset: $12.5 - 14 = -1.5$ dB

Typical Offset Sources:

- PCB losses: -0.5 dB
- Matching network: -0.8 dB
- Connector: -0.2 dB
- **Total: -1.5 dB ✓**

Apply Offset in Device Tree

Add `tx-power-offset = <-2>;` to compensate
(rounded to nearest integer)

Configure TX Power Offset

```
&spi2 {  
    lr1120: lr1120@0 {  
        compatible = "semtech,lr1120";  
        reg = <0>;  
  
        // Calibration results  
        tx-power-offset = <-2>; // dB compensation  
  
        // Antenna characteristics  
        antenna-gain = <2>; // 2 dBi antenna  
  
        // Regulatory limit (EU868)  
        tx-power-max = <14>; // EIRP limit  
  
        // Per-band calibration (optional)  
        tx-power-offset-868 = <-2>;  
        tx-power-offset-915 = <-1>;  
    };  
};
```

Frequency Accuracy Calibration

Why Frequency Matters:

- LoRaWAN requires ± 20 ppm
- Crystal tolerance varies
- Temperature affects frequency
- Regulatory compliance

Measurement:

- 1 Transmit CW at 868 MHz
- 2 Measure on spectrum analyzer
- 3 Calculate error in Hz
- 4 Apply correction

Example:

- Target: 868.100 MHz
- Measured: 868.095 MHz
- Error: -5 kHz
- PPM: $-5000/868100000 = -5.8$ ppm

```
1 &l r1120 {  
2     freq-error-hz =  
3     <-5000>;  
    };
```

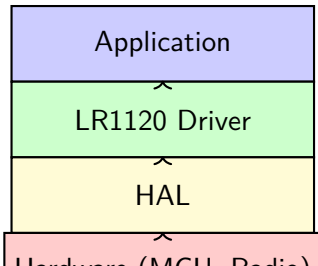
Radio driver will compensate by adding 5 kHz to all frequencies.

Hardware Abstraction Layer (HAL)

What is HAL?

HAL provides a standard interface between radio driver and hardware-specific operations:

- SPI communication
- GPIO control (reset, busy, IRQ)
- Timing functions (delays, timestamps)
- Power management



← Port this layer

HAL SPI Interface

```
// Initialize SPI bus
int hal_spi_init(const struct device *spi_dev);

// SPI write transaction
int hal_spi_write(const struct device *spi_dev,
                  const uint8_t *data,
                  size_t len);

// SPI read transaction
int hal_spi_read(const struct device *spi_dev,
                 uint8_t *data,
                 size_t len);

// SPI write-then-read (typical for radio)
int hal_spi_write_read(const struct device *spi_dev,
                       const uint8_t *tx_data,
                       uint8_t *rx_data,
                       size_t len);
```

HAL GPIO Interface

```
// Configure GPIO pin
int hal_gpio_init(const struct gpio_dt_spec *gpio,
                  gpio_flags_t flags);

// Set GPIO output level
int hal_gpio_set(const struct gpio_dt_spec *gpio,
                 int value);

// Read GPIO input level
int hal_gpio_get(const struct gpio_dt_spec *gpio);

// Configure GPIO interrupt
int hal_gpio_interrupt_configure(
    const struct gpio_dt_spec *gpio,
    gpio_flags_t flags,
    gpio_callback_handler_t handler);

// Enable/disable GPIO interrupt
int hal_gpio_interrupt_enable(
```

HAL Timing Interface

```
// Delay functions
void hal_delay_us(uint32_t microseconds);
void hal_delay_ms(uint32_t milliseconds);

// Timestamp functions (for ranging, etc.)
uint32_t hal_get_timestamp_us(void);
uint64_t hal_get_timestamp_ms(void);

// Timer callbacks
typedef void (*hal_timer_callback_t)(void *context);

int hal_timer_start(uint32_t timeout_ms,
                   hal_timer_callback_t callback,
                   void *context);

int hal_timer_stop(void);
```

Listing 8: Timing HAL Functions

Example: Porting to STM32

```
// File: hal_stm32_spi.c

#include <zephyr/drivers/spi.h>
#include "lr11xx_hal.h"

static const struct device *spi_dev;

int hal_spi_init(void) {
    spi_dev = DEVICE_DT_GET(
        DT_NODELABEL(spi1));
    if (!device_is_ready(spi_dev)) {
        return -ENODEV;
    }
    return 0;
}

int hal_spi_write_read(const uint8_t *tx_data,
                       uint8_t *rx_data,
                       size_t len) {
```

HAL Porting Checklist

1 SPI Driver

- Initialize SPI peripheral
- Configure speed (16 MHz max for LR1120)
- Implement write, read, write-read
- Handle chip select (CS) control

2 GPIO Driver

- Configure RESET, BUSY, DIO1 pins
- Implement GPIO read/write
- Configure DIO1 interrupt (rising edge)

3 Timing

- Microsecond delays for SPI timing
- Millisecond delays for radio operations
- Timestamp functions for ranging

4 Power Management

- Optional: implement radio power control
- Optional: low-power sleep integration

Verification Steps:

① Basic Communication

- Read LR1120 chip version register
- Expected: 0x02 (LR1121 v0x02)

② GPIO Control

- Toggle RESET pin, verify radio resets
- Check BUSY pin reads correctly
- Verify DIO1 interrupt triggers

③ Radio Operations

- Configure radio for TX
- Transmit test packet
- Receive packet in RX mode

④ Timing Accuracy

- Verify delay functions with oscilloscope
- Check timestamp accuracy for ranging

Steps to Integrate Custom Board

1 Create Board Definition

- `boards/arm/myboard/myboard.dts`
- `boards/arm/myboard/Kconfig.board`
- `boards/arm/myboard/Kconfig.defconfig`
- `boards/arm/myboard/board.cmake`

2 Configure Peripherals

- SPI for radio
- I2C for sensors
- UART for console
- GPIO for LEDs, buttons

3 Add HAL Implementation

- Implement required HAL functions
- Test with simple radio operations

4 Calibrate

- TX power calibration
- Frequency calibration
- Update device tree

Board Definition Example

```
/dts-v1/;  
#include <nordic/nrf52840.dtsi>  
  
/ {  
    model = "My Custom LoRaWAN Board";  
    compatible = "vendor,myboard";  
  
    chosen {  
        zephyr,console = &uart0;  
        zephyr,sram = &sram0;  
        zephyr,flash = &flash0;  
    };  
  
    aliases {  
        lora0 = &lr1120;  
        led0 = &led0;  
    };  
};
```

Common Integration Issues

SPI Communication Failures

Symptoms: Wrong data read, timeout errors

Solutions:

- Check SPI clock speed (max 16 MHz for LR1120)
- Verify SPI mode (LR1120 uses mode 0)
- Check CS pin control (active low)
- Verify MOSI/MISO not swapped

GPIO Interrupt Not Firing

Symptoms: Radio events not received

Solutions:

- Verify DIO1 pin configured as input
- Check interrupt edge (should be rising edge)
- Enable interrupt in NVIC
- Check GPIO voltage levels (3.3V vs 1.8V)

Power Consumption Optimization

Hardware Considerations:

- Use DCDC regulator (not LDO)
- Disable unused peripherals
- Configure GPIO pull resistors
- Use low-power TCXO
- Optimize antenna matching

Typical Current:

Mode	Current
Deep sleep	1.5 μ A
Idle (RTC)	5 μ A
RX (SF7)	15 mA
TX +14dBm	120 mA
TX +22dBm	140 mA

Firmware Optimizations:

- Enable CONFIG_PM
- Use lowest SF possible
- Increase TX interval
- Batch sensor readings
- Use LBM low-power API

Battery Life Estimate:

1000 mAh battery, 1 TX/hour:

$$\text{Life} = \frac{1000}{0.005 + \frac{24 \times 0.15}{3600}} \approx 200 \text{ days}$$

Lab 5.1: Device Tree Configuration

Objective

Create custom device tree overlay to configure LR1120 with different GPIO pins.

Tasks:

- 1 Create `myboard.overlay` with custom pin assignments
- 2 Configure SPI2 for LR1120 (CS on P0.10, different from default)
- 3 Assign RESET to P0.20, BUSY to P0.21, DIO1 to P0.22
- 4 Add I2C sensor configuration (BME680)
- 5 Build and flash firmware
- 6 Verify radio communication works with new pins

Deliverables:

- `myboard.overlay` file
- Serial output showing successful radio initialization
- Test transmission and reception

Lab 5.2: TX Power Calibration

Objective

Measure actual TX power output and calibrate for regulatory compliance.

Tasks:

- 1 Build test firmware with CW transmission mode
- 2 Connect board to spectrum analyzer via attenuator
- 3 Measure output power at +14 dBm setting
- 4 Calculate offset (actual - configured)
- 5 Update device tree with `tx-power-offset`
- 6 Rebuild and verify corrected output power
- 7 Test at multiple power levels (0, 10, 14, 20 dBm)

Deliverables:

- Calibration measurements table
- Updated device tree with offset
- Verification measurements showing compliance

Lab 5.3: Custom Board HAL Port

Objective

Port HAL to a different board (e.g., STM32 or ESP32).

Tasks:

- 1 Create new board definition in boards/
- 2 Implement HAL SPI functions for target MCU
- 3 Implement HAL GPIO functions
- 4 Implement HAL timing functions
- 5 Test basic radio communication (read chip version)
- 6 Run LoRaWAN join and uplink test
- 7 Measure and document any timing or performance differences

Deliverables:

- Complete board definition files
- HAL implementation source code
- Test results showing successful LoRaWAN operation
- Porting guide documenting challenges and solutions

Course 5 Summary

Key Takeaways

- Device tree enables hardware configuration without code changes
- TX power calibration ensures regulatory compliance
- HAL provides abstraction for different MCU platforms
- Proper GPIO and SPI configuration critical for reliability
- Power optimization requires both hardware and firmware considerations

Next Course:

- Course 6: Advanced Features
- FUOTA (Firmware Update Over-The-Air)
- LoRaWAN Relay networks
- Certification and testing
- Production deployment

Additional Resources

Documentation:

- Zephyr Device Tree Guide:
<https://docs.zephyrproject.org/latest/guides/dts/>
- LR1120/LR1121 Datasheet (Semtech)
- AN1200.22: LR11xx Hardware Design Guide
- Regional regulatory documents (ETSI EN 300 220, FCC Part 15, etc.)

Example Code:

- `boards/arm/xiao_nrf54l15/` (reference board)
- `drivers/loralr11xx_hal_zephyr.c` (HAL implementation)
- `samples/loralr11xx_tx_power_calibration/`

Lab Solutions:

- `doc/training/Lab_Solutions.md`

Thank You!

Questions?

Next: Course 6 - Advanced Features

Contact: `usp-support@example.com`